

Olá, estudante.
A seguir, você dará continuidade ao desenvolvimento da sua solução através dos campos específicos para a resolução dos 3 desafios propostos, lembrando que eles se complementam.

Nome: Rafael Damasceno

RGM: 36644595

Documentação:
Revisite a documentação do projeto desenvolvido na PIT I, faça as atualizações e melhorias necessárias de acordo com os pontos explicitados no material teórico. Ao terminar os ajustes, suba a documentação em um repositório GIT.
Link do repositório: https://github.com/Rafatec13/MaintFlow

Título do projeto: MaintFlow — Sistema simples de gestão de manutenção

Problema: equipes de manutenção registram equipamentos, falhas e intervenções em controles dispersos, o que dificulta a priorização, o acompanhamento do atendimento e a rastreabilidade das ações executadas.

Objetivo geral: desenvolver uma aplicação web simples para cadastrar equipamentos, acompanhar ordens de serviço, consultar o histórico e visualizar um painel básico, com execução local no Windows e hospedagem no PythonAnywhere.

Usuários previstos: pessoas responsáveis pelo registro e acompanhamento da manutenção, com acesso mediante login.

Funcionalidades: login e encerramento de sessão; cadastro de equipamentos; abertura e acompanhamento de ordens; mudanças de status; histórico por equipamento; e painel básico. Os cinco colegas avaliaram seus roteiros em 13/10/2026, sem falhas relatadas.



Requisitos: Python com Flask, HTML e CSS, banco SQLite, validação no servidor e proteção por sessão. Execução local no Windows e hospedagem no PythonAnywhere. Os resultados relatados pelos colegas abrangem acesso, manutenção e navegação.

Modelagem: Usuário, Equipamento, Ordem de Serviço e Histórico. Relações 1:N: equipamento–ordens, ordem–histórico e usuário–histórico. Identificadores e chaves estrangeiras estão no SQLite. Modelo conceitual, lógico e dicionário físico constam no documento complementar Modelagem_MaintFlow.md.

Situação do projeto: código, README e instruções publicados no GitHub. Três testes automatizados passaram com Python 3.13.1 no PythonAnywhere. Cinco avaliações de usuários em 13/10/2026 tiveram resultados positivos nos roteiros executados. Evidências individuais a anexar.

Codificação:

Na Tabela a seguir insira as informações referentes ao desenvolvimento do código do *front-end* e *back-end*.

Linguagem do *Back-end*	Python com Flask; código publicado no GitHub
Banco de Dados	SQLite em arquivo; esquema definido no código
Hospedagem	PythonAnywhere; aplicação Flask com banco SQLite
Plataforma	Aplicação web em HTML e CSS; execução local ou hospedada
Modo de Codificação	(x) Tradicional ( ) Low-code ( ) No-code
***Link* do repositório no GitHub com os códigos abertos**	https://github.com/Rafatec13/MaintFlow
***Link* da solução em funcionamento**	https://rafaeldamascenotech.pythonanywhere.com/
***Link* do vídeo narrado (no mínimo 5 min)**	*https://youtu.be/-N0dFLF6bL8*

Testes da Solução

Escolha 5 colegas para testar sua aplicação e preencha a tabela a seguir com as informações obtidas:

Nome do colega: Leando S.	Data do teste: 13/10/2026
O que deverá ser testado: Roteiro 1 — Acesso: tentar senha incorreta, entrar com credenciais válidas, abrir o painel, sair e tentar acessar uma página protegida. Esperado: rejeitar senha incorreta e exigir login após sair.	
Resultado e correção: O sistema rejeitou a senha incorreta e permitiu acesso com usuário válido. Após sair, a página protegida exigiu novo login. Nenhuma falha identificada; não houve necessidade de correção.	
Funcionalidade não testada: nenhuma dentro do roteiro atribuído ao colega; todas as etapas previstas foram executadas.	

Nome do colega: Silvio Dagnaldo	Data do teste: 13/10/2026
O que deverá ser testado: Roteiro 2 — Equipamentos: cadastrar tag, nome e setor; consultar o cadastro; tentar tag repetida e campo obrigatório vazio; atualizar a página. Esperado: manter o cadastro e rejeitar dados inválidos.	
Resultado e correção: O cadastro manteve os dados após atualizar a página. O sistema impediu tags repetidas e campos obrigatórios vazios. Nenhuma falha identificada; não houve necessidade de correção.	
Funcionalidade não testada: nenhuma dentro do roteiro atribuído ao colega; todas as etapas previstas foram executadas.	

Nome do colega: Lucas	Data do teste: 13/10/2026
O que deverá ser testado: Roteiro 3 — Ordens: selecionar um equipamento, informar descrição e prioridade, abrir uma ordem e consultar a lista; tentar descrição vazia. Esperado: vincular a ordem corretamente e rejeitar campo vazio.	
Resultado e correção: A abertura de ordens vinculou corretamente o equipamento e a prioridade. O sistema impediu o cadastro sem descrição. Nenhuma falha identificada; não houve necessidade de correção.	

Nome do colega: Lucas	Data do teste: 13/10/2026
Funcionalidade não testada: nenhuma dentro do roteiro atribuído ao colega; todas as etapas previstas foram executadas.	

Nome do colega: Leonardo	Data do teste: 13/10/2026
O que deverá ser testado: Roteiro 4 — Histórico: iniciar e concluir uma ordem; abrir o histórico do equipamento. Esperado: status Aberta, em andamento e concluída, com registro de abertura e alterações no histórico.	
Resultado e correção: As ordens avançaram de Aberta para Em andamento e concluída. O histórico registrou as alterações corretamente. Nenhuma falha identificada; não houve necessidade de correção.	
Funcionalidade não testada: nenhuma dentro do roteiro atribuído ao colega; todas as etapas previstas foram executadas.	

Nome do colega: Luciano	Data do teste: 13/10/2026
O que deverá ser testado: Roteiro 5 — Painel: conferir totais de equipamentos e ordens por status; abrir e avançar uma ordem; atualizar o painel e navegar pelas páginas. Esperado: contagens coerentes e navegação compreensível.	
Resultado e correção: O painel apresentou os totais de equipamentos e ordens, atualizados após alterações. A navegação entre as páginas funcionou corretamente. Nenhuma falha identificada; não houve necessidade de correção.	
Funcionalidade não testada: nenhuma dentro do roteiro atribuído ao colega; todas as etapas previstas foram executadas.	

Laudo de Qualidade

Insira a seguir o laudo de qualidade do sistema, apontando os erros e as correções. Não esqueça de coletar as evidências para inseri-las no laudo:

Em 13/10/2026, três testes automatizados passaram no PythonAnywhere com Python 3.13.1, cobrindo acesso, fluxo de manutenção e validações. Cinco colegas também avaliaram login, equipamentos, ordens, histórico e painel, conforme os resultados registrados nas tabelas.

Erros e correções: nenhuma falha foi relatada pelos cinco colegas nos roteiros executados em 13/10/2026. Assim, não houve necessidade de correções ou retestes decorrentes dessas avaliações. O resultado se limita aos cenários testados.

Evidências: código em https://github.com/Rafatec13/MaintFlow e demonstração em https://youtu.be/-N0dFLF6bL8. A captura dos testes automatizados foi apresentada no acompanhamento. Essa captura e as evidências dos cinco colegas devem ser anexadas ao relatório ou disponibilizadas por link.

Conclusão: nos três testes automatizados e nos cinco roteiros executados pelos colegas, o MaintFlow apresentou resultados positivos, sem falhas relatadas. O sistema atendeu aos cenários avaliados. As evidências dos testes devem acompanhar o relatório para comprovar os resultados.

Vídeo da Solução atualizada

Após levantar os *feedbacks* e executar as correções necessárias e pertinentes, grave um vídeo de até 5 minutos apresentando as modificações realizadas no sistema.

***Link* para o vídeo**	https://youtu.be/-N0dFLF6bL8